

COMP3308/3608 Deep Learning Notes

Author

K. Yu

2026

Contents

1	Overview	3
1.1	High-level idea	3
1.2	More specific definitions	3
1.3	Why deep neural networks?	4
2	Problems with traditional Backpropagation Networks	4
3	Autoencoders	4
3.1	Basic definition	4
3.2	Stacked Autoencoders	6
3.3	Layer-wise pre-training	7
3.4	Fine-tuning	7
3.5	Why pre-training helps	8
3.6	Other types of Autoencoders	8
4	Convolutional Neural Networks	9
4.1	Motivation	9
4.2	Convolution operation	10
4.3	Pooling, Normalisation, and Dropout	11
4.4	Overall CNN Architecture	13
4.5	Applications of CNNs	14
5	Recurrent Neural Networks (Not Examinable)	15
5.1	Motivation	15
5.2	Basic RNN structure	15
5.3	Sequence modelling tasks	16
5.4	Backpropagation Through Time	16
5.5	Vanishing gradients and long-term dependencies	17
5.6	LSTM and GRU	17

6	Attention Mechanism and Transformers (Not Examinable)	18
6.1	Motivation	18
6.2	Attention as weighted information retrieval	19
6.3	Query, Key, and Value	19
6.4	Self-Attention	20
6.5	Multi-Head Attention	21
6.6	Transformer Architecture	22
6.7	Positional Encoding	22
6.8	Encoder, Decoder, and common Transformer Models	23
6.9	Why Transformers became important	24
7	Why did Deep Learning improve so dramatically?	25
8	Cautions about Deep Learning	25
8.1	Deep learning is not always the best solution	25
8.2	Common sense matters	25
8.3	Interpretability	26

1 Overview

1.1 High-level idea

Deep learning is a branch of machine learning that studies **large neural networks** capable of learning useful representations from data.

Compared with classical machine learning, deep learning is especially useful when:

- the input data is unstructured, e.g. image, speech, text;
- the dataset is large;
- manually designing good features is difficult;
- hierarchical structure exists in the data.

Examples include image search, machine translation, speech recognition, face detection, self-driving cars, and medical image analysis.

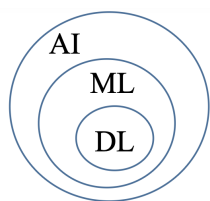


Figure 1: The relationship is roughly $\text{Deep Learning} \subset \text{Machine Learning} \subset \text{Artificial Intelligence}$.

1.2 More specific definitions

Deep learning can mean several related things:

1. **A neural network with more than one hidden layer**, usually known as deep neural networks (DNN).
2. **Automatic feature learning**. Instead of relying on hand-designed feature engineering, the model learns useful internal representations directly from data.
3. **Layer-wise representation learning**. Lower layers learn simple features, while higher layers combine them into more abstract features.

For example, in object detection:

pixels $\rightarrow$ edges $\rightarrow$ parts $\rightarrow$ objects.

This is the main reason deep learning is often described as learning **hierarchical feature representations**.

1.3 Why deep neural networks?

A classical result called **Cybenko's Theorem** says that a neural network with one hidden layer can approximate any continuous function arbitrarily well, under suitable assumptions.

However, this is only an **existence theorem**. It says much a network exists, but it does not tell us how to estimate its parameters efficiently nor whether the network is practical; whether it generalises well; whether it is the most compact representation.

A DNN may represent some functions much more efficiently than a shallow network. In practice, depth often gives better feature learning and better generalisation, especially for unstructured data such as image, audio, and text.

2 Problems with traditional Backpropagation Networks

Traditional fully connected neural networks trained by backpropagation have several limitations:

1. **Slow training.** Many epochs may be required.
2. **Too many parameters.** Fully connected layers create a very large number of weights.
3. **Sensitive to initialisation.** Bad initial weights can make training slow or unstable.
4. **Vanishing and exploding gradients.** In DNNs, gradients can become extremely small (vanishing) or uncontrollably large (exploding) as they are propagated back through layers, making it difficult to update weights effectively.
5. **Large labelled datasets are required.** In many real applications, labelled data is expensive or difficult/inefficient to obtain.
6. **Local Minima and Plateaus.** Backpropagation uses gradient descent, which can get stuck in local minima (sub-optimal solutions) instead of finding the global minimum.
7. **Feature engineering is still needed.** Classical models often rely heavily on manually designed features.

Deep learning architectures were developed partly to address these problems.

3 Autoencoders

3.1 Basic definition

Definition 3.1.1 (Autoencoders). An **autoencoder** is a neural network trained to reconstruct its input.

Suppose we have unlabelled input vectors $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)} \in \mathbb{R}^d$. For an autoencoder, the target output is the input itself, i.e. $\mathbf{y}^{(i)} = \mathbf{x}^{(i)}$. The network learns a mapping

$$\mathbf{x}^{(i)} \mapsto \mathbf{h}^{(i)} \mapsto \hat{\mathbf{x}}^{(i)},$$

where $\hat{\mathbf{x}}^{(i)} \in \mathbb{R}^d$ is the reconstructed output. The goal is

$$\hat{\mathbf{x}}^{(i)} \approx \mathbf{x}^{(i)}.$$

As shown in Figure 2, a typical autoencoder has fewer hidden neurons than input neurons, so the hidden layer is forced to learn a compressed representation.

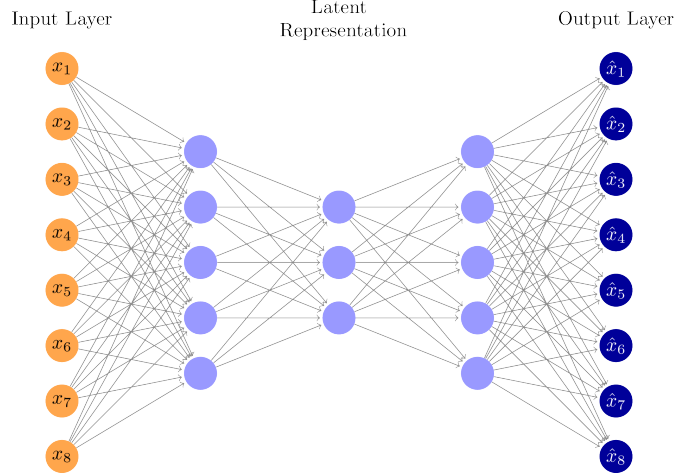


Figure 2: The architecture of autoencoder neural networks.

Encoder, decoder, and reconstruction loss

The autoencoder can be split into two parts:

$$\mathbf{h} = f_{\theta}(\mathbf{x}) \quad \text{and} \quad \hat{\mathbf{x}} = g_{\phi}(\mathbf{h}).$$

Here, f_{θ} is the **encoder**; g_{ϕ} is the **decoder**; θ and ϕ are sets of model parameters.

A common reconstruction loss is:

$$\mathcal{L}_{\text{rec}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2.$$

For binary input data, another possible reconstruction loss is binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^d \left[x_j^{(i)} \log x_j^{(i)} + (1 - x_j^{(i)}) \log(1 - x_j^{(i)}) \right].$$

Main idea of autoencoders

The important part of an autoencoder is usually not the output, but the hidden representation $\mathbf{h}$. For example, a 10×10 grayscale image can be flattened into a 100-dimensional vector $\mathbf{x} \in \mathbb{R}^{100}$. If the hidden layer has 50 neurons, then $\mathbf{h}$ is a compressed 50-dimensional representation of the image. This compressed vector can then be used for:

- dimensionality reduction;
- image compression;
- feature extraction;
- input to another classifier;
- pre-training DNNs.

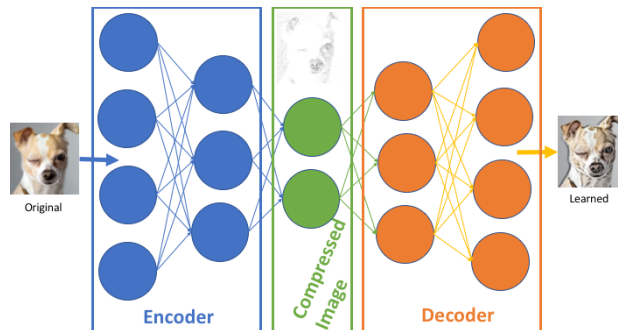


Figure 3: An image compression autoencoder.

Traditional applications

Autoencoders have historically been used for:

1. **Dimensionality reduction.** The hidden vector $\mathbf{h}$ gives a lower-dimensional representation of the input.
2. **Data compression.** The encoder compresses the input, and the decoder reconstructs it.
3. **Image compression.** Pixel values can be compressed into a smaller hidden representation.
4. **Encryption-like encoding.** If the encoder weights are W_1 and decoder weights are W_2 , then W_1 can encode the input and W_2 can decode it.

3.2 Stacked Autoencoders

Motivation

DNNs can be hard to train from random initialisation. One older strategy is to use autoencoders to initialise the weights layer by layer. This is called **unsupervised pre-training**.

The idea is:

1. train the first layer using an autoencoder;
2. use the learned hidden representation as input to the next autoencoder;
3. repeat for deeper layers;

4. finally attach the supervised output layer;
5. fine-tune the whole network using backpropagation.

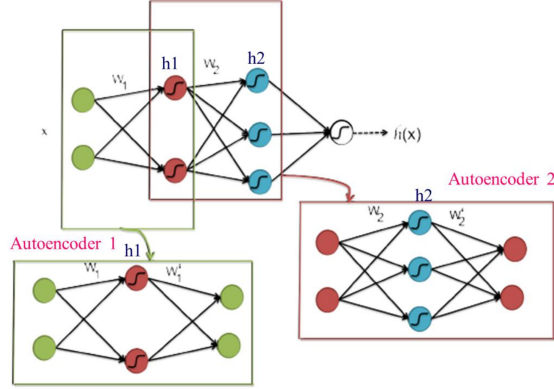


Figure 4: Autoencoders as initialisation method for DNN.

3.3 Layer-wise pre-training

Suppose we want to train a DNN with two hidden layers:

$$\mathbf{x} \mapsto \mathbf{h}_1 \mapsto \mathbf{h}_2 \mapsto \mathbf{y}.$$

First, train Autoencoder 1:

$$\mathbf{x} \mapsto \mathbf{h}_1 \mapsto \hat{\mathbf{x}}.$$

This learns weights W_1 from $\mathbf{x}$ to $\mathbf{h}_1$. After training, W_1 is kept, while the decoder weights are discarded. Then compute:

$$\mathbf{h}_1 = f_1(\mathbf{x}).$$

Next, train Autoencoder 2:

$$\mathbf{h}_1 \mapsto \mathbf{h}_2 \mapsto \hat{\mathbf{h}}_1.$$

This learns weights W_2 from $\mathbf{h}_1$ to $\mathbf{h}_2$. Again, W_2 is kept and the decoder weights are discarded. The pre-trained network now has:

$$\mathbf{x} \xrightarrow{W_1} \mathbf{h}_1 \xrightarrow{W_2} \mathbf{h}_2.$$

3.4 Fine-tuning

After unsupervised pre-training, the model is fine-tuned using labelled data.

1. **Pre-training.** Train one autoencoder per layer using unlabelled data.
2. **Fine-tuning step 1.** Train the final supervised layer using labelled data.
3. **Fine-tuning step 2.** Train the whole network using backpropagation.

The supervised loss can be written as:

$$\mathcal{L}_{\text{sup}} = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, \hat{y}^{(i)})$$

where ℓ is a suitable loss function, such as cross-entropy for classification.

3.5 Why pre-training helps

Unsupervised pre-training can help because it gives the DNN a better starting point. Empirically, pre-trained networks often:

- converge faster;
- generalise better;
- are less likely to get stuck in poor local minima;
- learn more meaningful early-layer features.

The intuition is that each layer learns useful structure in the data before the final supervised task begins. For image data, as shown in Figure 5, early layers may learn strokes or edges, middle layers may learn parts, and higher layers may learn whole objects or digits.

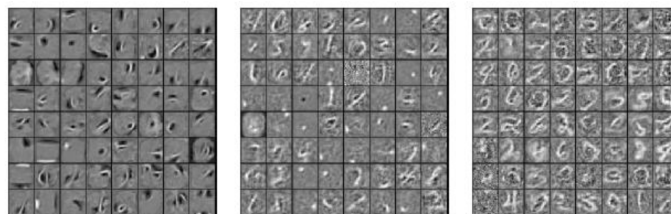


Figure 5: MNIST stacked autoencoder feature hierarchy. First layer: strokes. Second layer: digit parts. Third layer: whole digits.

3.6 Other types of Autoencoders

Sparse autoencoder

A **sparse autoencoder** may have more hidden neurons than input dimensions. This means it is not necessarily compressing the input. Instead, sparsity constraints force most hidden neurons to be inactive for a given input.

The goal is to learn useful structure by encouraging only a small number of hidden units to activate. A sparse autoencoder may use a loss of the form

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{sparse}},$$

where $\mathcal{L}_{\text{rec}}$ measures reconstruction error; $\mathcal{L}_{\text{sparse}}$ penalises too many active hidden units; and λ controls the strength of the sparsity penalty.

Denoising autoencoder

A **denoising autoencoder** is trained to reconstruct the original input from a corrupted version. Let $\tilde{\mathbf{x}}$ be a noisy version of $\mathbf{x}$. The model learns

$$\tilde{\mathbf{x}} \mapsto \mathbf{h} \mapsto \hat{\mathbf{x}}$$

with loss

$$\mathcal{L}_{\text{denoise}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2.$$

This forces the model to learn robust features instead of simply copying the input.

This idea is related to dropout because both encourage robustness by preventing the network from relying too heavily on specific input components or neurons.

4 Convolutional Neural Networks

4.1 Motivation

A fully connected network is inefficient for image and geospatial data.

For example, a 100×100 images has $10 \times 10 = 10,000$ input pixels. If each hidden neuron is fully connected to every input pixel, then each hidden neuron has 10,000 weights, plus a bias. This becomes computationally expensive very quickly.

Convolutional neural networks (CNN) solve this problem by using:

1. **local connectivity**;
2. **weight sharing**;
3. **convolutional kernels**;
4. **pooling**.

CNNs are designed to recognise visual patterns directly from pixels with minimal preprocessing. They are especially effective for images because nearby pixels are strongly related.

Local connectivity

In a fully connected network, each neuron connects to every neuron in the previous layer. Whereas in a CNN, each neuron only connects to a small local region of the input, called its **receptive field**. For image data, this means a neuron may only look at a small patch of pixels, such as 3×3 or 5×5 . This greatly reduces the number of parameters.

Weight sharing

CNNs further reduce parameters using **weight sharing**. Instead of learning different weights for every local region, CNNs apply the same small set of weights across many positions in the image. This shared set of weights is called a **kernel** or **filter**.

For example, if the kernel is

$$K = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix},$$

the same 3×3 weights are applied to many different image patches. This allows the same feature detector to detect the same pattern in different parts of the image. So the key idea is that the kernel is a learned feature detector.

4.2 Convolution operation

For a single-channel image X and a $r \times s$ kernel K , the convolution output at position (i, j) is:

$$[S]_{i,j} = \sum_{a=1}^r \sum_{b=1}^s [K]_{a,b} [X]_{i+a-1, j+b-1}$$

In practice, deep learning libraries such as PyTorch, TensorFlow often implement cross-correlation rather than mathematically flipped convolution, but the intuition is the same: slide the kernel over the image and compute local weighted sums.

The output S is called a **feature map**. High values in the feature map mean the kernel strongly detected its feature at that location.

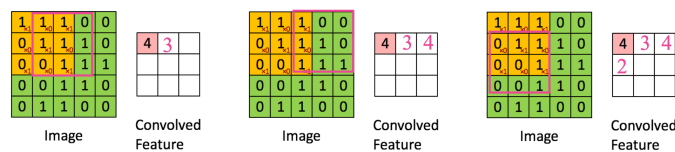


Figure 6: An example of sliding window convolution.

Multiple kernels and feature maps

A convolutional layer usually contains many kernels. If the layer has C_{out} kernels, then it produces C_{out} feature maps. For example,

$$\begin{aligned} \text{Input image} &\mapsto \text{Kernel 1} \mapsto \text{Feature map 1,} \\ \text{Input image} &\mapsto \text{Kernel 2} \mapsto \text{Feature map 2.} \end{aligned}$$

Each feature map detects a different type of feature. For an RGB image, the input has 3 channels, i.e. a tensor

$$X \in \mathbb{R}^{H \times W \times 3}.$$

A kernel must also extend across the input channels:

$$K \in \mathbb{R}^{r \times s \times 3}$$

For a layer with C_{out} kernels, the full kernel tensor has shape:

$$K \in \mathbb{R}^{r \times s \times C_{in} \times C_{out}}.$$

4.3 Pooling, Normalisation, and Dropout

Max pooling

A convolutional layer is often followed by a pooling layer. The most common type is **max pooling**. For a small region R , max pooling computes:

$$\text{MaxPool}(R) = \max\{\mathbf{z} \in R\}.$$

For example, if a 2×2 region is:

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix},$$

then $R = \{1, 2, 3, 4\}$ and the max pooling outputs:

$$\text{MaxPool}(R) = 4.$$

Pooling reduces the spatial size of the feature maps and summarises local information.

Translation invariance

Max pooling gives some robustness to small shifts. If a feature moves slightly within the pooling region, the maximum value may remain similar. Therefore, the pooled output can stay stable even when the input shifts. This is useful because images often contain objects at slightly different positions.

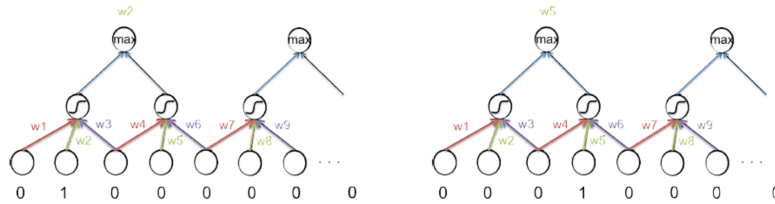


Figure 7: Input images (1-dim), each with a white dot which got shifted 2 pixels to the right.

Strictly speaking, pooling gives local translation invariance or translation robustness, not perfect invariance across the whole image.

Local Contrast Normalisation

Sometimes a max-pooling layer is followed by **Local Contrast Normalisation**. The idea is to normalise local neuron outputs, i.e.

$$z'_i = \frac{z_i - \mu_{\text{local}}}{\sigma_{\text{local}}}.$$

This helps the model become more robust to brightness changes. However, in modern CNNs batch normalisation and layer normalisation are more common.

Batch Normalisation (not examinable)

Batch Normalisation (BN) is another normalisation technique commonly used in modern DNNs. Unlike LCN, which normalises a neuron using nearby local neurons, BN normalises neurons using statistics computed from a mini-batch.

For a mini-batch of neurons $z^{(1)}, z^{(2)}, \dots, z^{(m)}$, BN first computes the mini-batch mean and variance:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m z^{(i)}, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (z^{(i)} - \mu_B)^2.$$

Then each neuron is normalised as:

$$\hat{z}^{(i)} = \frac{z^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

where $\epsilon > 0$ is a small correction constant used to avoid division by zero. Finally, BN applies a learnable rescaling and shifting step:

$$\tilde{z}^{(i)} = \gamma \hat{z}^{(i)} + \beta,$$

where γ and β are trainable (estimated) parameters.

The purpose of BN is to make the distribution of neurons more stable during training. This often makes neural networks easier and faster to train, and can also provide some regularisation. In modern CNNs, BN is usually more common than LCN. However, for COMP3308/3608, BN is not assessable.

Dropout

Dropout is a regularisation technique used to reduce overfitting. During training, each neuron is randomly dropped with probability p . Dropping a neuron means temporarily setting its activation to zero.

If $\mathbf{h}$ is a hidden representation and $\mathbf{m}$ is a binary mask, then dropout can be written as

$$\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h},$$

where each component of $\mathbf{m}$

$$m_j = \begin{cases} 0 & \text{with probability } p, \\ 1 & \text{with probability } 1 - p. \end{cases}$$

Here $\odot$ means elementwise multiplication.

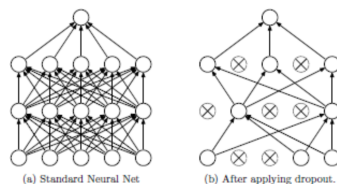


Figure 8: Dropout prevents the network from overfit.

Dropout forces the model not to rely too heavily on individual neurons. This makes the network more robust. During testing, dropout is turned off.

4.4 Overall CNN Architecture

Typical structure

A typical CNN has repeated blocks of

Convolution $\rightarrow$ Activation $\rightarrow$ Pooling $\rightarrow$ Optional Normalisation.

After several such blocks, the network usually ends with one or more fully connected layers. For classification out of K classes, the final layer often outputs class probabilities through:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) = \begin{bmatrix} \frac{\exp(z_1)}{\sum_{j=1}^K \exp(z_j)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{j=1}^K \exp(z_j)} \end{bmatrix} \in (0, 1)^K.$$

The classification loss is usually cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N y^{(i)} \log \hat{y}^{(i)}.$$

Backpropagation in CNNs

CNNs are still trained using backpropagation. The difference is that the architecture has constraints:

- local connectivity;
- shared weights;
- convolutional kernels;
- pooling operations.

For shared weights, the same kernel parameter is used at multiple spatial locations. During backpropagation, each use contributes to the gradient. The total update for a shared weight is the sum of gradient contributions from all locations where that weight was used.

Update rule:

one shared kernel weight $\leftarrow$ sum of all gradient contributions involving that weight.

This keeps the kernel consistent across the whole image.

4.5 Applications of CNNs

LeNet-5

LeNet-5 is an early CNN architecture developed by Yann LeCun and collaborators for handwritten digit recognition.

Its architecture includes:

- 2 convolutional layers;
- 2 max-pooling layers;
- 3 fully connected layers.

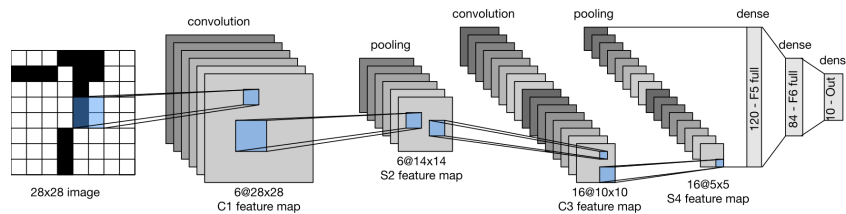


Figure 9: Architecture of LeNet-5.

The important point is that LeNet-5 uses local connectivity instead of full connectivity, making it much more suitable for image data.

ImageNet and AlexNet-style CNNs

ImageNet was a major turning point for deep learning in computer vision. The ImageNet dataset contains large-scale labelled images (1.2 million) across many categories (1000 classes).

Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton trained a deep CNN with:

- 5 convolutional layers;
- 3 max-pooling layers;
- 3 fully connected layers; around 60 million weights;
- around 650,000 neurons.

Important techniques included:

- dropout;

- ReLU activation

$$\text{ReLU}(x) = \max(0, x),$$

- efficient GPU implementation.

The model achieved a major improvement in ImageNet classification, helping trigger the modern deep learning boom.

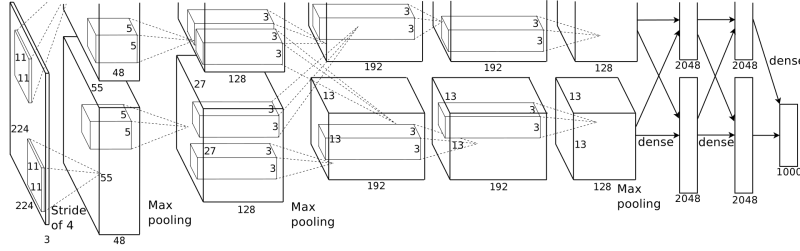


Figure 10: AlexNet architecture.

5 Recurrent Neural Networks (Not Examinable)

5.1 Motivation

CNNs are mainly designed for image-like data, where local spatial structure is important. However, many real-world datasets are **sequential** rather than spatial.

Examples include:

- text, where a sentence is a sequence of words;
- speech, where an audio signal changes over time;
- time series, such as stock prices or sensor readings;
- videos, where frames appear in temporal order.

For these tasks, the order of the input matters. A standard feedforward neural network does not naturally remember previous inputs, so it is not well suited for modelling sequences.

A **Recurrent Neural Network** (RNN) is designed to process sequential data by maintaining a hidden state that acts like a memory of previous inputs.

5.2 Basic RNN structure

Suppose the input sequence is $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$, where $\mathbf{x}_t$ is the input vector at time step t . An RNN updates its hidden state recursively:

$$\mathbf{h}_t = f(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b})$$

where W_x maps the current input to the hidden state, W_h maps the previous hidden state into the current hidden state, and f is a nonlinear activation function, such as $\tanh$.

The key idea is that $\mathbf{h}_t$ depends on both the current input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$. Therefore, the network can carry information forward through the sequence. The output may be computed as

$$\hat{\mathbf{y}} = g(W_y \mathbf{h}_t + \mathbf{c})$$

where g is an output activation function, such as softmax for classification.

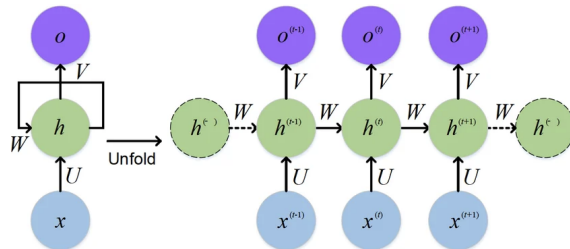


Figure 11: A simple unfolded RNN.

5.3 Sequence modelling tasks

RNNs can be used for different types of sequence tasks.

For **many-to-one** tasks, the network reads the whole input sequence and outputs one prediction. For example:

sentence $\rightarrow$ sentiment label.

For **many-to-many** tasks, the network outputs a sequence. For example:

English sentence $\rightarrow$ French sentence.

For **one-to-many** tasks, the network receives one input and generates a sequence. For example:

image representation $\rightarrow$ image caption.

The main point is that RNNs are useful when the prediction depends on order and context.

5.4 Backpropagation Through Time

RNNs are trained using a variant of backpropagation called **Backpropagation Through Time** (BPTT).

The idea is to unfold the RNN across time steps and then apply backpropagation to the unfolded computational graph. If the supervised target at time t is $\mathbf{y}_t$, a typical sequence loss is:

$$\mathcal{L} = \sum_{t=1}^T \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t),$$

where ℓ is a suitable loss function, such as cross-entropy.

For example, in sequence classification, the model may only use the final hidden state $\mathbf{h}_T$, so the loss simply becomes

$$\mathcal{L} = \ell(y, \hat{y}).$$

5.5 Vanishing gradients and long-term dependencies

A major problem with basic RNNs is the **vanishing gradient problem**.

When BPTT is applied through many time steps, gradients may become very small. As a result, the model struggles to learn long-range dependencies.

For example, in the sentence:

The cats that lived near the old house were hungry.

the word "cats" helps determine that the verb should be "were", but the dependency spans several words.

A basic RNN may struggle to preserve this information over many time steps. This is why more advanced recurrent architectures were developed.

5.6 LSTM and GRU

Two important RNN variants are **Long Short-Term Memory** (LSTM) and **Gated Recurrent Unit** (GRU). Both introduce **gates** that control how information is stored, updated, and forgotten.

The purpose of these gates is to help the model preserve useful information over longer sequences and reduce the vanishing gradient problem.

At a high level:

- an **LSTM** has input, forget, and output gates;
- a **GRU** has reset and update gates;
- both are designed to model long-term dependencies better than a basic RNN.

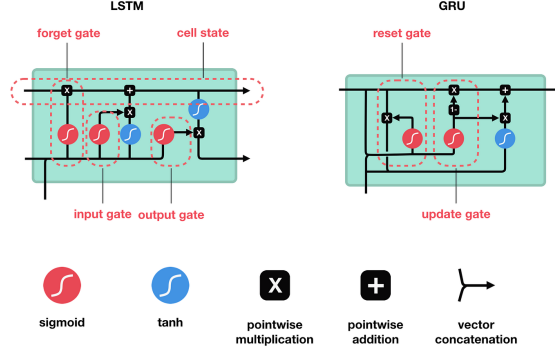


Figure 12: Architecture of LSTM and GRU.

6 Attention Mechanism and Transformers (Not Examinable)

6.1 Motivation

Although LSTM and GRU improve basic RNNs, they still process sequences step by step. This creates two main problems.

1. The model may still struggle with very long-range dependencies. Even with gates, information from early time steps must pass through many intermediate hidden states before reaching later time steps.
2. RNN-style models are hard to parallelise. Since the hidden state at time step t depends on the hidden state at time step $t - 1$, the model must process the sequence in order.

Attention mechanisms were introduced to address these limitations. The key idea is simple: *Instead of forcing the model to compress all previous information into a single hidden state, let the model directly look at the most relevant parts of the input sequence.*

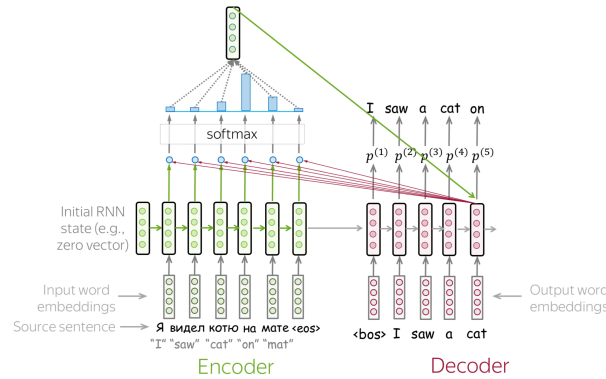


Figure 13: A sequence-to-sequence model with attention.

For example, in machine translation, when translating a word in the output sentence, the model should focus more on the relevant words in the input sentence rather than treating all input words equally.

6.2 Attention as weighted information retrieval

Suppose we have a sequence of hidden vectors $\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T$. When predicting an output, the model computes an attention score for each hidden vector. A larger score means that the corresponding time step is more relevant.

The scores are then converted into weights using softmax: $\alpha_1, \alpha_2, \dots, \alpha_T$, where each α_t is between 0 and 1, and all attention weights sum to 1.

The context vector is a weighted average of the hidden vectors:

$$\mathbf{c} = \sum_{t=1}^T \alpha_t \mathbf{h}_t.$$

So the context vector contains more information from important positions and less information from irrelevant positions.

The main intuition is: *Attention lets the model dynamically decide where to focus*. This is especially useful for text, because the meaning of a word often depends on other words that may be far away in the sentence.

6.3 Query, Key, and Value

Modern attention is usually described using three vectors:

- **Query:** what we are looking for;
- **Key:** what each position contains;
- **Value:** the information stored at each position.

For a given query $\mathbf{q}$, the model compares it with each key $\mathbf{k}_t$ to compute a similarity score. These scores are normalised into attention weights. The final output is a weighted average of the value vectors.

Intuitively:

- the query asks a question;
- the keys decide which positions are relevant;
- the values provide the actual information to be combined.

A common version is **scaled dot-product attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Here:

- Q is the query matrix;
- K is the key matrix;
- V is the value matrix;
- d_k is the dimension of the key vectors;
- the factor $\sqrt{d_k}$ prevents dot products from becoming too large.

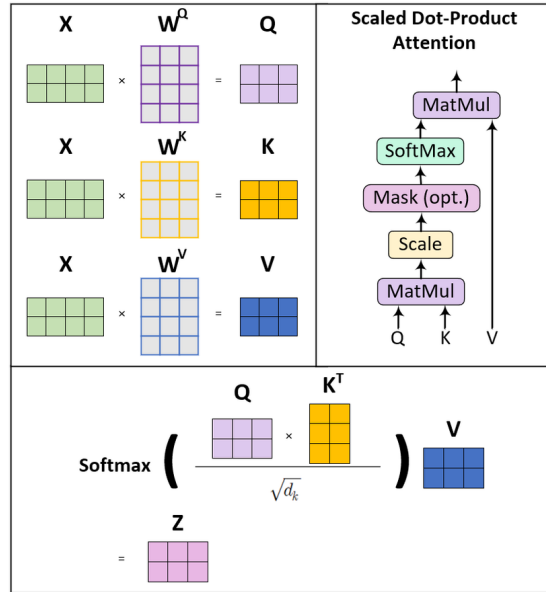


Figure 14: Diagram of the query, key, value, and attention-mechanism.

6.4 Self-Attention

Self-attention is attention **applied within a single sequence**. For example, in the sentence: "The animal did not cross the street because it was tired." The word "it" refers to "the animal". A self-attention layer can allow the representation of "it" to directly use information from "animal", even though they are separated by several words.

In self-attention, **every token can attend to every other token** in the same sequence. Therefore, each word representation is updated using information from the whole sentence.

This is different from RNNs:

- RNNs pass information sequentially through hidden states;
- self-attention connects all positions directly.

This makes self-attention much better at modelling **long-range dependencies** and easier to parallelise.

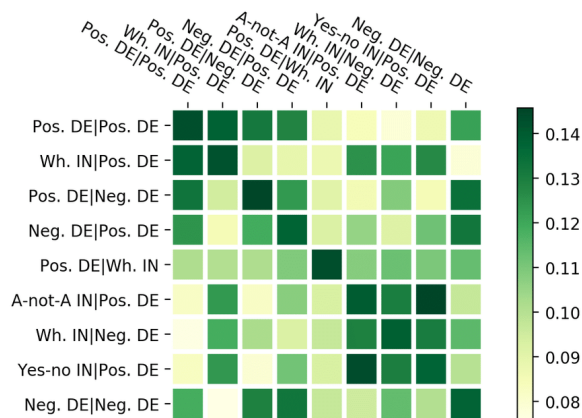


Figure 15: Self-attention heatmap for a sentence, each word attends to other words.

6.5 Multi-Head Attention

Instead of using only one attention mechanism, Transformers use multi-head attention.

The idea is to run several attention mechanisms in **parallel**. Each attention head can focus on different types of relationships.

For example, in a sentence, different heads may learn to capture:

- subject-verb relationships;
- pronoun reference;
- nearby phrase structure;
- long-distance semantic relationships.

The outputs of all heads are then concatenated and transformed by a linear layer.

The main intuition is: *Multi-head attention allows the model to look at the sequence from multiple perspectives at the same time.*

This makes the representation richer than using a single attention head.

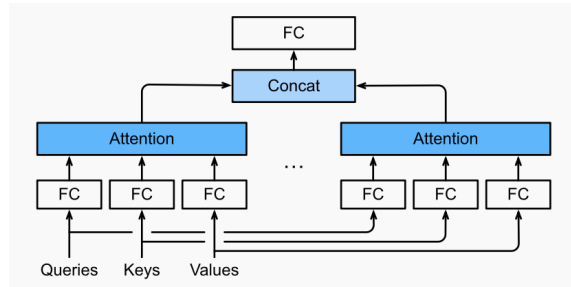


Figure 16: Multi-head attention, where multiple heads are concatenated then linearly transformed.

6.6 Transformer Architecture

The Transformer is a neural network architecture based mainly on self-attention. Unlike RNNs, it does not process tokens one by one. Instead, it processes the whole sequence in parallel.

A standard Transformer block contains:

1. multi-head self-attention;
2. residual connection and layer normalisation;
3. feedforward neural network;
4. another residual connection and layer normalisation.

The self-attention layer allows tokens to exchange information. The feedforward layer then transforms each token representation independently.

At a high-level, one Transformer block does:

```
input token representations
-> self-attention
-> add & normalise
-> FFN
-> add & normalise
-> output token representations
```

6.7 Positional Encoding

Since Transformers process all tokens in parallel, they do not automatically know the order of the sequence.

For example, the phrases: "dog bites man" and "man bites dog" contain the same words, but their meanings are very different because the **order** is different.

To solve this, Transformers add positional information to the token embeddings. This is called **positional encoding**.

The input representation becomes:

`token embedding + positional encoding`

The token embedding tells the model **what** the word is. The positional encoding tells the model **where** the word is.

Without positional encoding, the Transformer would treat the input more like an unordered set of tokens.

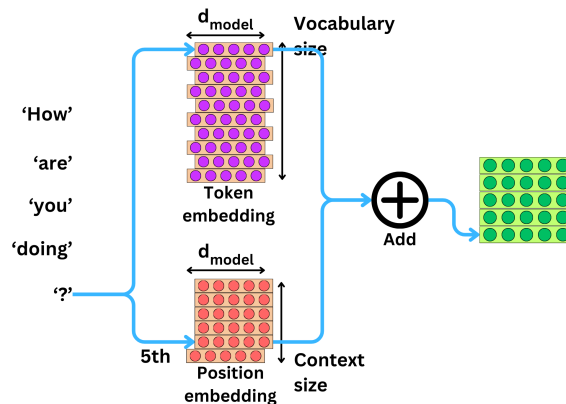


Figure 17: Token embeddings plus positional encodings before entering the Transformer.

6.8 Encoder, Decoder, and common Transformer Models

The original Transformer architecture has an encoder and a decoder.

- The encoder reads the input sequence and produces contextual representations.
- The decoder generates the output sequence one token at a time, using both previous output tokens and the encoder output.

Different modern models use different parts of this architecture.

- **BERT** is mainly encoder-based. It is good for understanding tasks such as classification, sentiment analysis, and question answering.
- **GPT**-style models are mainly decoder-based. They are good for generation tasks because they predict the next token given previous tokens.

Encoder-decoder models are useful for **sequence-to-sequence** tasks such as translation and summarisation.

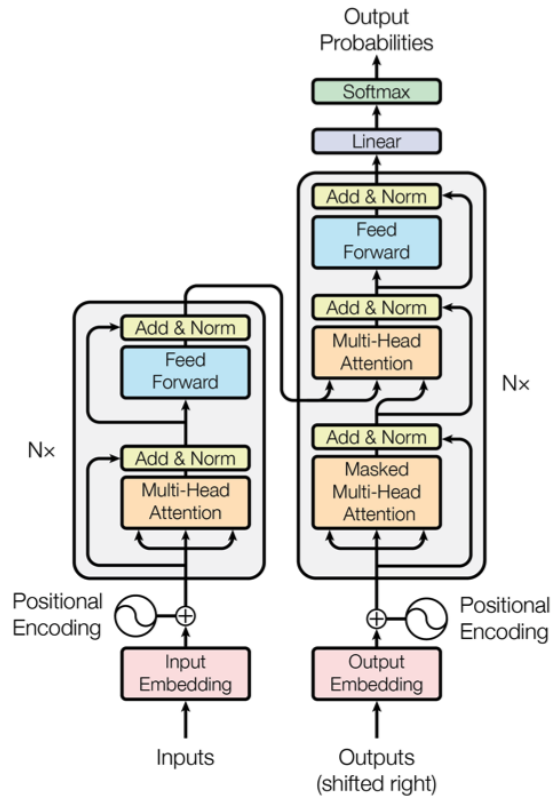


Figure 18: Transformer model architecture.

6.9 Why Transformers became important

Transformers became dominant in modern deep learning because they have several advantages:

1. They model long-range dependencies well. Any token can directly attend to any other token.
2. They are easier to parallelise than RNNs. Since the whole sequence can be processed at once, training is much faster on GPUs and TPUs.
3. They scale well. Large Transformer models can be trained on massive datasets and then adapted to many downstream tasks.
4. They work not only for text but also for images, audio, code, and multimodal data.

For example:

- In NLP, Transformers are used for language models and machine translation;
- in Computer Vision, Transformers use **patch embedding** to transform images into tokens;
- in multimodal learning, models can combine text, images, and other inputs.

7 Why did Deep Learning improve so dramatically?

Although many ideas behind neural networks existed for decades, deep learning became dramatically more successful because of several factors:

1. **More computational power.** GPUs made it possible to train large neural networks efficiently.
2. **Larger datasets.** Datasets such as ImageNet provided millions of labelled examples.
3. **Better training techniques.** Important techniques include dropout, convolution and weight sharing, autoencoder pre-training, better activation functions such as ReLU, and improved visualisation of learned features.
4. **Better software ecosystems.** Libraries such as TensorFlow, Keras, PyTorch made deep learning easier to implement and scale.

8 Cautions about Deep Learning

8.1 Deep learning is not always the best solution

DNNs are powerful, but they are not a universal solution. For some problems, simpler methods may perform as well or better.

Classical models can be preferable when:

- the dataset is small;
- interpretability is important;
- the signal is simple;
- training resources are limited;
- a strong baseline already works well.

Key takeaway: Always compare with simple and meaningful baselines.

A sophisticated neural networks is not automatically better than a simple rule, a random forest, gradient-boosted tree, or a well-designed baseline.

8.2 Common sense matters

Deep learning should be used when the problem structure and data justify it.

Good problem understanding > Blindly applying complex models.

8.3 Interpretability

DNNs are often difficult to interpret. This is a disadvantage in high-stakes domains such as:

- medical diagnosis;
- credit assessment;
- legal or administrative decisions;
- automated decisions affecting people.

In such settings, we may need not only a prediction, but also an explanation of why the prediction was made.

Deep learning models are often less interpretable than models such as decision trees or linear models. Therefore, interpretability and explainability are important research areas.